C3M3: Peer Reviewed Assignment

Outline:

The objectives for this assignment:

1. Implement kernel smoothing in R and interpret the results.
2. Implement smoothing splines as an alternative to kernel estimation.
3. Implement and interpret the loess smoother in R.
4. Compare and contrast nonparametric smoothing methods.

General tips:

1. Read the questions carefully to understand what is being asked.
2. This work will be reviewed by another human, so make sure that you are clear and concise in what your explanations and answers.

In [1]:
```
# Load Required Packages
library(ggplot2)
library(mgcv)
```

```
Loading required package: nlme

This is mgcv 1.8-31. For overview type 'help("mgcv-package")'.
```

Problem 1: Advertising data

The following dataset containts measurements related to the impact of three advertising medias on sales of a product, P. The variables are:

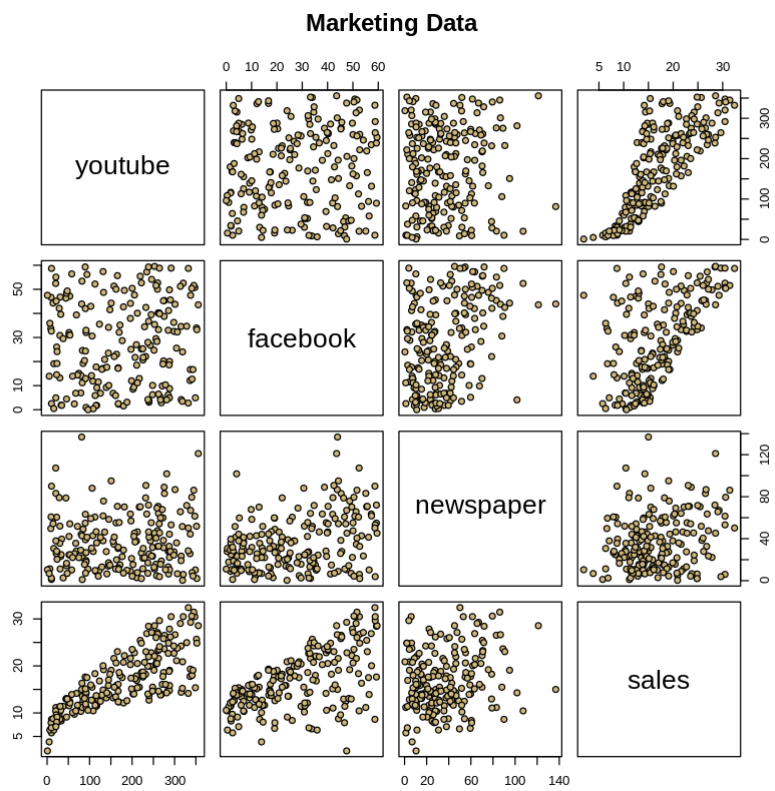
- `youtube` : the advertising budget allocated to YouTube. Measured in thousands of dollars;
- `facebook` : the advertising budget allocated to Facebook. Measured in thousands of dollars; and
- `newspaper` : the advertising budget allocated to a local newspaper. Measured in thousands of dollars.
- `sales` : the value in the i^{th} row of the sales column is a measurement of the sales (in thousands of units) for product P for company i.

The advertising data treat "a company selling product P" as the statistical unit, and "all companies selling product P" as the population. We assume that the $n = 200$ companies in the dataset were chosen at random from the population (a strong assumption!).

First, we load the data, plot it, and split it into a training set (`train_marketing`) and a test set (`test_marketing`).

In [2]:
```
# Load in the data
marketing = read.csv("marketing.txt", sep="")
summary(marketing)
pairs(marketing, main = "Marketing Data", pch = 21,
      bg = c("#CFB87C"))
```

```
    youtube          facebook       newspaper          sales      
 Min.   :  0.84   Min.   : 0.00   Min.   :  0.36   Min.   : 1.92  
 1st Qu.: 89.25   1st Qu.:11.97   1st Qu.: 15.30   1st Qu.:12.45  
 Median :179.70   Median :27.48   Median : 30.90   Median :15.48  
 Mean   :176.45   Mean   :27.92   Mean   : 36.66   Mean   :16.83  
 3rd Qu.:262.59   3rd Qu.:43.83   3rd Qu.: 54.12   3rd Qu.:20.88  
 Max.   :355.68   Max.   :59.52   Max.   :136.80   Max.   :32.40  
```



In [3]:
```
set.seed(1771) #set the random number generator seed.
n = floor(0.8 * nrow(marketing)) #find the number corresponding to 80% of the data
index = sample(seq_len(nrow(marketing)), size = n) #randomly sample indicies to be included in the training set

train_marketing = marketing[index, ] #set the training set to be the randomly sampled rows of the dataframe
test_marketing = marketing[-index, ] #set the testing set to be the remaining rows
dim(test_marketing) #check the dimensions
dim(train_marketing) #check the dimensions
```

40 · 4

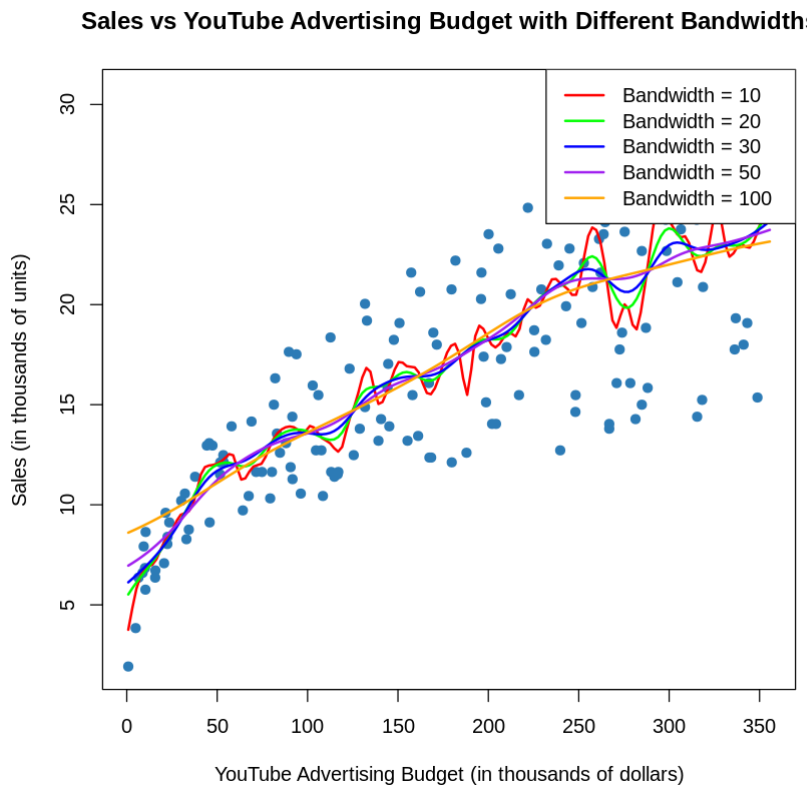
160 · 4

1.(a) Working with nonlinearity: Kernel regression

Note that the relationship between `sales` and `youtube` is nonlinear. This was a problem for us back in the first course in this specialization, when we modeled the data as if it were linear. For now, let's just focus on the relationship between `sales` and `youtube`, omitting the other variables (future lessons on generalized additive models will allow us to bring back other predictors).

Using the `train_marketing` set, plot `sales` (response) against `youtube` (predictor), and then fit and overlay a kernel regression. Experiment with the bandwidth parameter until the smooth looks appropriate, or comment why no bandwidth is ideal. Justify your answer.

In [4]:
```
# Different bandwidths to try
bandwidths <- c(10, 20, 30, 50, 100)

# Plot sales vs youtube
plot(train_marketing$youtube, train_marketing$sales,
     xlab = "YouTube Advertising Budget (in thousands of dollars)",
     ylab = "Sales (in thousands of units)",
     main = "Sales vs YouTube Advertising Budget with Different Bandwidths",
     pch = 19, col = "#2C7BB6")

# Add kernel regression lines for each bandwidth
colors <- c("red", "green", "blue", "purple", "orange")
for (i in 1:length(bandwidths)) {
  kernel_reg <- ksmooth(train_marketing$youtube, train_marketing$sales,
                        kernel = "normal", bandwidth = bandwidths[i])
  lines(kernel_reg$x, kernel_reg$y, col = colors[i], lwd = 2)
}

legend("topright", legend = paste("Bandwidth =", bandwidths), col = colors, lwd = 2)
```

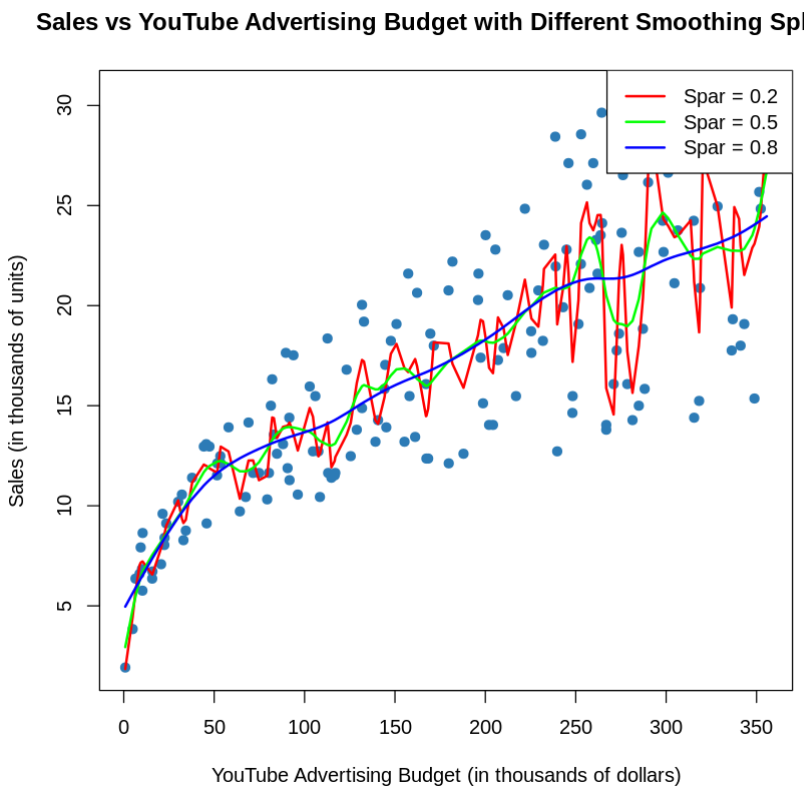


Bandwidth = 30 appears to be the most appropriate, providing a balance between capturing the underlying trend and smoothing out noise. However, the ideal bandwidth depends on the specific goals of the analysis.

1.(b) Working with nonlinearity: Smoothing spline regression

Again, using the `train_marketing` set, plot `sales` (response) against `youtube` (predictor). This time, fit and overlay a smoothing spline regression model. Experiment with the smoothing parameter until the smooth looks appropriate. Explain why it's appropriate and justify your answer.

In [5]:
```r
# Different smoothing parameters to try
spar_values <- c(0.2, 0.5, 0.8)

# Plot sales vs youtube
plot(train_marketing$youtube, train_marketing$sales,
     xlab = "YouTube Advertising Budget (in thousands of dollars)",
     ylab = "Sales (in thousands of units)",
     main = "Sales vs YouTube Advertising Budget with Different Smoothing Splines",
     pch = 19, col = "#2C7BB6")

# Add smoothing spline lines for each spar value
colors <- c("red", "green", "blue")
for (i in 1:length(spar_values)) {
  spline_model <- smooth.spline(train_marketing$youtube, train_marketing$sales, spar = spar_values[i])
  lines(spline_model, col = colors[i], lwd = 2)
}

legend("topright", legend = paste("Spar =", spar_values), col = colors, lwd = 2)
```



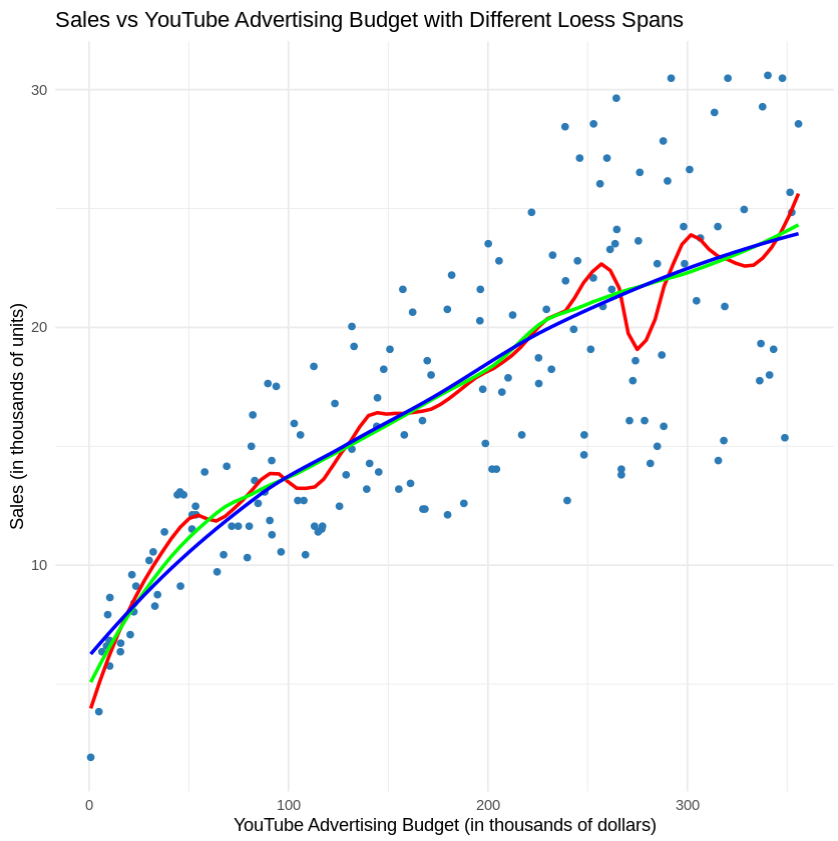
The chosen spar value of 0.5 is appropriate because it offers a smooth yet flexible representation of the data, avoiding both overfitting and underfitting, which is crucial for making reliable predictions and understanding the relationship between sales and YouTube advertising budget.

1.(c) Working with nonlinearity: Loess

Again, using the `train_marketing` set, plot `sales` (response) against `youtube` (predictor). This time, fit and overlay a loess regression model. You can use the `loess()` function in a similar way as the `lm()` function. Experiment with the smoothing parameter (`span` in the `geom_smooth()` function) until the smooth looks appropriate. Explain why it's appropriate and justify your answer.

In [6]:
```
# Experiment with different span values
span_values <- c(0.2, 0.5, 0.8)

# Plot sales vs youtube with different loess spans
ggplot(train_marketing, aes(x = youtube, y = sales)) +
  geom_point(color = "#2C7BB6") +
  geom_smooth(method = "loess", span = 0.2, color = "red", se = FALSE) +
  geom_smooth(method = "loess", span = 0.5, color = "green", se = FALSE) +
  geom_smooth(method = "loess", span = 0.8, color = "blue", se = FALSE) +
  labs(title = "Sales vs YouTube Advertising Budget with Different Loess Spans",
       x = "YouTube Advertising Budget (in thousands of dollars)",
       y = "Sales (in thousands of units)") +
  theme_minimal()
```

```
`geom_smooth()` using formula 'y ~ x'

`geom_smooth()` using formula 'y ~ x'

`geom_smooth()` using formula 'y ~ x'
```



This span value provides a good general-purpose model that is likely to perform well in both fitting the current data and generalizing to new, unseen data.

1.(d) A prediction metric

Compare the models using the mean squared prediction error (MSPE) on the `test_marketing` dataset. That is, calculate the MSPE for your kernel regression, smoothing spline regression, and loess model, and identify which model is best in terms of this metric.

Remember, the MSPE is given by

$$MSPE = \frac{1}{k}\sum_{i=1}^{k}\left(y_i^\star - \hat{y}_i^\star\right)^2$$

where $y_i^\star$ are the observed response values in the test set and $\hat{y}_i^\star$ are the predicted values for the test set (using the model fit on the training set).

*Note that `ksmooth()` orders your designated `x.points`. Make sure to account for this in your MSPE calculation.

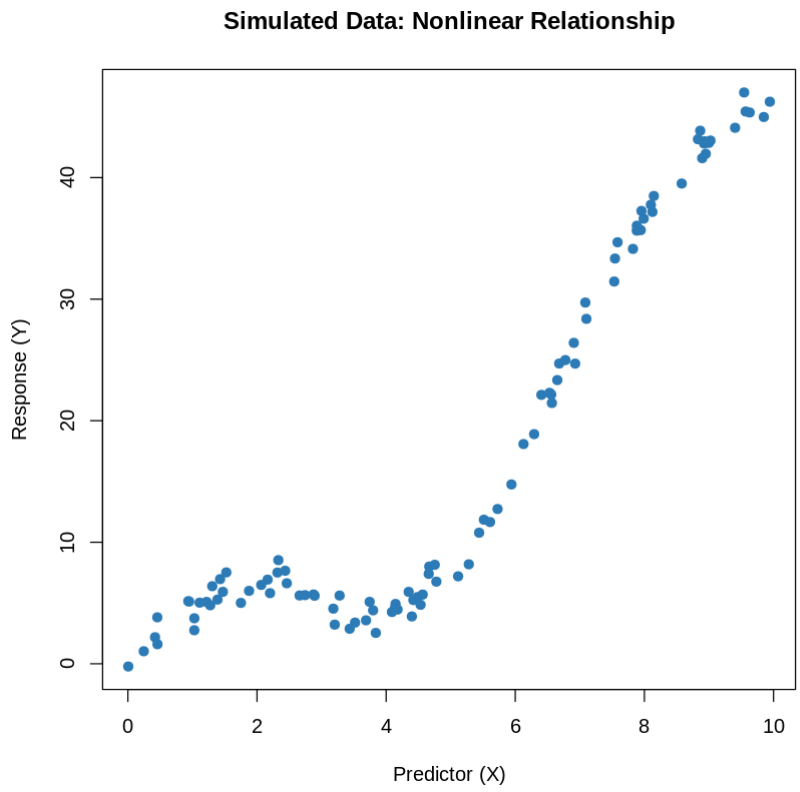
In [7]:
```
# Kernel Regression
bw <- 30  # Example bandwidth used previously
kernel_reg <- ksmooth(train_marketing$youtube, train_marketing$sales, kernel = "normal", bandwidth = bw, x.points = test_market
ing$youtube)
pred_kernel <- kernel_reg$y[order(order(kernel_reg$x))]  # Reorder predictions
mspe_kernel <- mean((test_marketing$sales - pred_kernel)^2)
print(paste("MSPE for Kernel Regression:", mspe_kernel))

# Smoothing Spline Regression
spline_model <- smooth.spline(train_marketing$youtube, train_marketing$sales, spar = 0.5)  # Using spar = 0.5 as an example
pred_spline <- predict(spline_model, test_marketing$youtube)$y
mspe_spline <- mean((test_marketing$sales - pred_spline)^2)
print(paste("MSPE for Smoothing Spline Regression:", mspe_spline))

# Loess Regression
loess_model <- loess(sales ~ youtube, data = train_marketing, span = 0.5)  # Using span = 0.5 as an example
pred_loess <- predict(loess_model, newdata = test_marketing)
mspe_loess <- mean((test_marketing$sales - pred_loess)^2)
print(paste("MSPE for Loess Regression:", mspe_loess))
```

```
[1] "MSPE for Kernel Regression: 65.1645770012368"
[1] "MSPE for Smoothing Spline Regression: 19.171706531712"
[1] "MSPE for Loess Regression: 18.1148323815482"
```

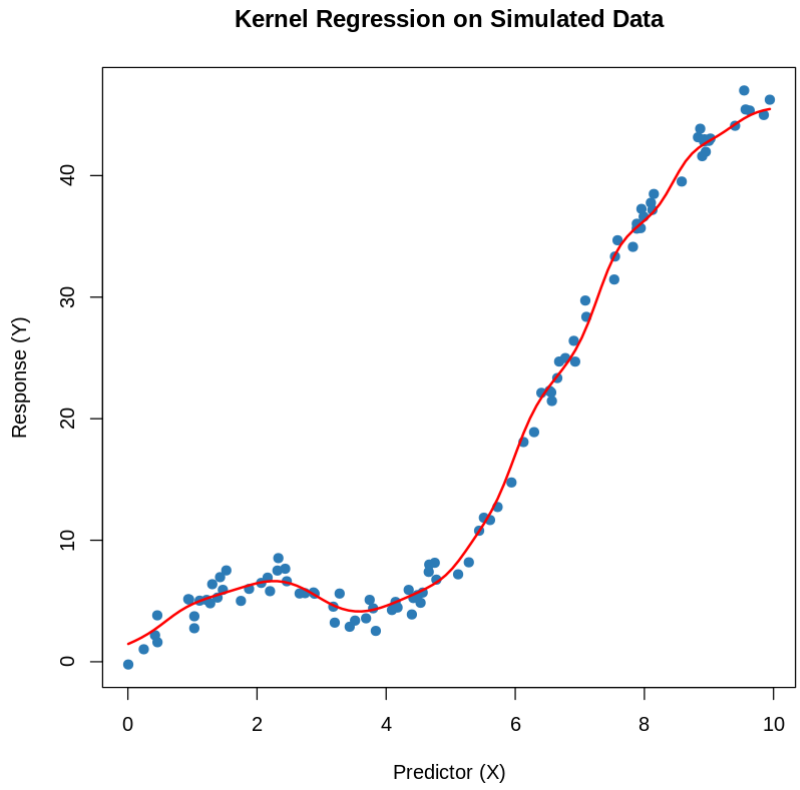
Problem 2: Simulations!

Simulate data (one predictor and one response) with your own nonlinear relationship. Provide an explanation of how you generated the data. Then answer the questions above (1.(a) - 1.(d)) using your simulated data.

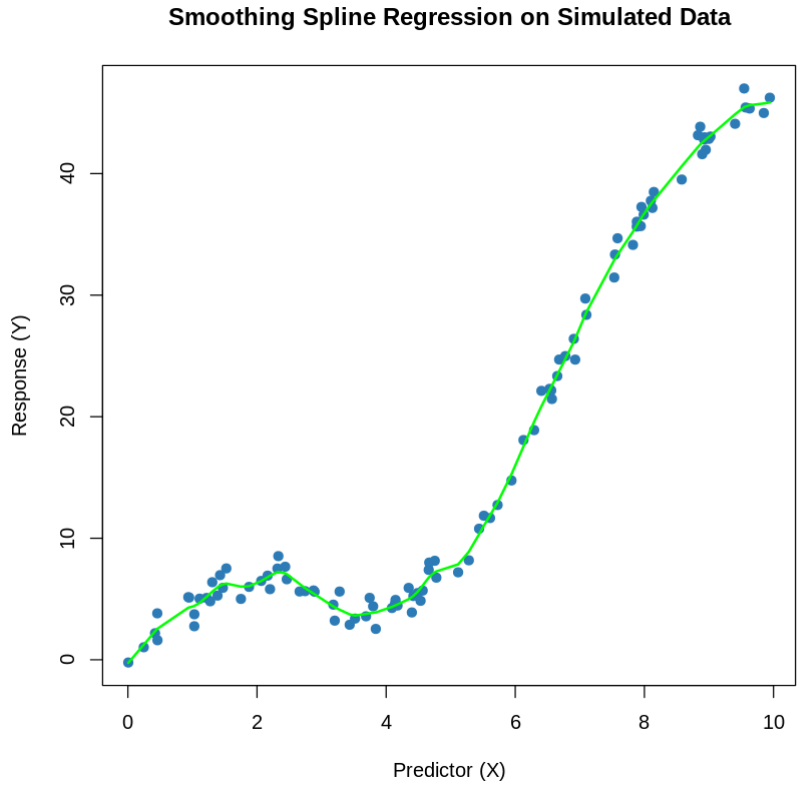
In [8]:
```r
#simulated data
set.seed(123)  # For reproducibility

# Simulate the predictor variable X
X <- runif(100, min = 0, max = 10)

# Simulate the response variable Y using the nonlinear relationship
epsilon <- rnorm(100, mean = 0, sd = 1)
Y <- 5*sin(X) + 0.5*X^2 + epsilon

# Create a data frame
simulated_data <- data.frame(X = X, Y = Y)

# Plot the simulated data
plot(simulated_data$X, simulated_data$Y,
     xlab = "Predictor (X)",
     ylab = "Response (Y)",
     main = "Simulated Data: Nonlinear Relationship",
     pch = 19, col = "#2C7BB6")
```



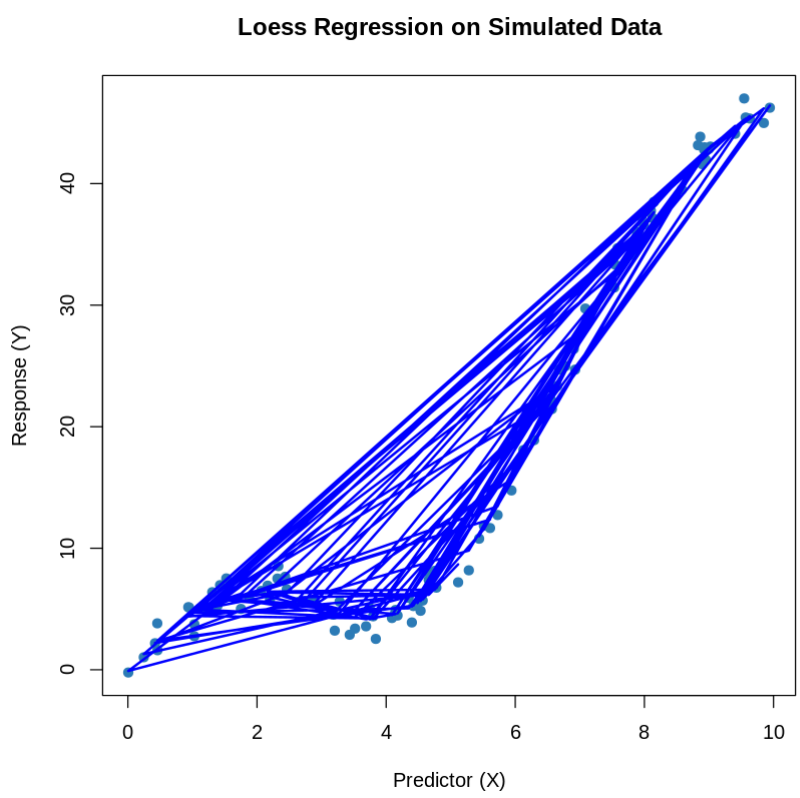
In [9]:
```r
#1.a
# Fit Kernel Regression
bw <- 1  # Experiment with bandwidth
kernel_reg_sim <- ksmooth(simulated_data$X, simulated_data$Y, kernel = "normal", bandwidth = bw)

# Plot the data and overlay the kernel regression
plot(simulated_data$X, simulated_data$Y, col = "#2C7BB6", pch = 19,
     xlab = "Predictor (X)", ylab = "Response (Y)", main = "Kernel Regression on Simulated Data")
lines(kernel_reg_sim, col = "red", lwd = 2)
```



In [10]:
```r
#1.b
# Fit Smoothing Spline
spline_model_sim <- smooth.spline(simulated_data$X, simulated_data$Y, spar = 0.5)  # Experiment with spar

# Plot the data and overlay the smoothing spline
plot(simulated_data$X, simulated_data$Y, col = "#2C7BB6", pch = 19,
     xlab = "Predictor (X)", ylab = "Response (Y)", main = "Smoothing Spline Regression on Simulated Data")
lines(spline_model_sim, col = "green", lwd = 2)
```



In [11]:
```r
#1.c
# Fit Loess Regression
loess_model_sim <- loess(Y ~ X, data = simulated_data, span = 0.5)  # Experiment with span

# Predict values for plotting
loess_pred <- predict(loess_model_sim)

# Plot the data and overlay the loess regression
plot(simulated_data$X, simulated_data$Y, col = "#2C7BB6", pch = 19,
     xlab = "Predictor (X)", ylab = "Response (Y)", main = "Loess Regression on Simulated Data")
lines(simulated_data$X, loess_pred, col = "blue", lwd = 2)
```



In [13]:
```
#1.d
# Simulate test data
set.seed(456)
X_test <- runif(100, min = 0, max = 10)
epsilon_test <- rnorm(100, mean = 0, sd = 1)
Y_test <- 5*sin(X_test) + 0.5*X_test^2 + epsilon_test

test_data <- data.frame(X = X_test, Y = Y_test)


kernel_reg_test <- ksmooth(simulated_data$X, simulated_data$Y, kernel = "normal", bandwidth = bw, x.points = test_data$X)
pred_kernel_test <- kernel_reg_test$y[order(order(kernel_reg_test$x))]
mspe_kernel_sim <- mean((test_data$Y - pred_kernel_test)^2)
print(paste("MSPE for Kernel Regression on Simulated Data:", mspe_kernel_sim))


pred_spline_test <- predict(spline_model_sim, test_data$X)$y
mspe_spline_sim <- mean((test_data$Y - pred_spline_test)^2)
print(paste("MSPE for Smoothing Spline on Simulated Data:", mspe_spline_sim))


pred_loess_test <- predict(loess_model_sim, newdata = test_data)
mspe_loess_sim <- mean((test_data$Y - pred_loess_test)^2)
print(paste("MSPE for Loess Regression on Simulated Data:", mspe_loess_sim))
```

```
[1] "MSPE for Kernel Regression on Simulated Data: 514.3649103197"
[1] "MSPE for Smoothing Spline on Simulated Data: 1.01823477942388"
[1] "MSPE for Loess Regression on Simulated Data: 1.01796643399973"
```

simulate a dataset with one predictor (X) and one response (Y) with a nonlinear relationship. Suppose the relationship between (X) and (Y) is defined as follows:

$$Y = 5\sin(X) + 0.5X^2 + \epsilon$$

where:

- (X) is uniformly distributed between 0 and 10.
- (\epsilon) is normally distributed noise with mean 0 and standard deviation 1.

In [ ]: